

REPORT

과제물 주제		LOTTO 복권 번호 추출 프로그램 작성
1차	2차	

성 명	정주호		학습과정명	C언어I	
학 번	225230001		교 · 강사명	한인	
전공명 / 학급명	컴퓨터공학 전공	A2	취득점수 / 교 · 강사확인		

1. 개요 및 문제 설명

1.1) 과제 목적

본 과제의 주제는 C언어와 Visual Studio를 이용해 'Lotto 복권 번호 추출' 기능을 수행하는 C 프로그램을 작성하는 것입니다. 기계가 번호 공을 하나씩 뽑는 실제 추첨 상황을, 컴퓨터가 1부터 45 사이의 난수를 생성하는 것으로 표현합니다.

생성된 번호는 배열 자료구조에 저장되며, 같은 번호가 다시 나오면 저장하지 않고 재추첨합니다. 최종 6개 번호는 오름차순으로 정렬하여 사람이 빠르게 확인할 수 있도록 출력합니다.

1.2) 핵심 요구사항

- I. rand() 함수를 이용하여, 1~45 사이의 임의의 정수를 중복 없이 6개 추출한다.
- II. 추출된 6개 숫자는 오름차순으로 정렬하여 출력한다.
- III. 위 과정을 5회 반복한다.
- IV. 위 프로그램을 기능별로 나누어 함수로 작성한다.
- V. 함수의 선언은 헤더 파일에, 함수의 정의는 별도의 소스 파일에 작성한다.

1.3) 핵심 개념

이론 수업을 통해 학습한 변수, 배열, 함수, 알고리즘 등의 핵심 개념들을 종합적으로 실습합니다. 배열은 6개의 번호를 저장하고, 함수는 생성·판정·정렬·출력의 역할을 나누어 담당합니다. 이러한 구조는 작은 프로그램에서도 유지보수 가능한 설계를 연습하는 데 의의가 있습니다.

1.4) 성공 기준

프로그램의 정상 동작을 판단하기 위한 네 가지 기준은 번호 범위, 중복 없음, 오름차순 정렬, 5회 출력입니다.

RULE 01	RULE 02	RULE 03	RULE 04
범위	중복 없음	오름차순	5회 출력
각 숫자는 1 이상 45 이하	한 회차 6개 숫자는 모두 다름	작은 숫자에서 큰 숫자 순서	추첨 과정을 총 5번 반복

검증 항목	확인 방법	통과 판단
범위 검사	각 배열 값이 1~45 안에 있는지 확인	벗어난 값 0건
중복 검사	정렬 후 인접 숫자가 같은지 확인	동일 숫자 0건
정렬 검사	이전 값 < 다음 값인지 확인	역순 또는 동률 0건
반복 검사	출력 회차의 개수를 확인	총 5개 결과 세트

2. 함수 설계 및 구현

2.1) 모듈형 파일 구조

함수 분리 | 기능을 먼저 선언하고, 정의한 뒤, main에서 호출합니다



프로그램을 역할별 파일로 나누고 함수를 기능별로 분리하여, 읽기 쉽고 수정하기 쉬운 구조로 설계하였습니다. 본 프로그램은 함수의 선언 → 정의 → 호출 순서가 분명하게 드러나도록 libLottoLogic.h, libLottoLogic.c, prgLottoMain.c의 세 파일로 구성하였습니다.

libLottoLogic.h는 프로그램에서 사용할 함수의 원형을 선언하는 파일이며, libLottoLogic.c는 번호 생성, 중복 판정, 선택정렬, 결과 출력 기능을 실제로 구현하는 파일입니다. 마지막으로 prgLottoMain.c의 main() 함수는 난수의 시작값을 설정하고, 필요한 기능을 순서대로 호출하여 프로그램 전체의 실행 흐름을 관리합니다.

초기 프로토타입에서는 getLottoNo() 함수 안에 번호 생성과 중복 판정 로직을 함께 작성하였습니다. 그러나 번호를 생성하는 기능과 이미 저장된 번호인지 확인하는 기능은 서로 다른 역할을 가진다고 판단하여, 중복 판정 기능을 isDuplicateNo() 함수로 분리하였습니다. 이를 통해 각 함수의 책임이 명확해지고, 추후 사용자 번호 입력이나 결과 비교 기능을 추가할 때에도 중복 판정 로직을 재사용할 수 있는 구조가 되었습니다.

이러한 파일 분리 방식은 main() 함수가 전체 실행 순서에 집중하고, LottoLogic 모듈이 실제 처리 과정을 담당하도록 역할을 구분한다는 점에서 의미가 있습니다. 따라서 특정 기능의 내부 구현이 바뀌더라도 전체 프로그램의 흐름은 안정적으로 유지할 수 있습니다.

파일	핵심 역할	함수 / 사용 방식
prgLottoMain.c	전체 순서를 제어	main() → 생성·정렬·출력 함수를 5회 호출
libLottoLogic.h	함수 사용법을 선언	getLottoNo(), isDuplicateNo(), sortArrAsc(), prnArr()
libLottoLogic.c	실제 기능(알고리즘)을 수행	번호 생성 / 중복 판정 / 선택정렬 / 화면 출력

2.2) 프로그램 동작 흐름

제어문 | 반복문과 조건문으로 주첨 과정을 제어



while(TRUE) : 중복이면 재추첨 | **for문 :** 6개 번호 저장 및 전체 과정 5회 반복

SEED 시드 생성	LOOP 반복문	ARRAY 배열
srand(time(0)) 실행 시점에 따라 서로 다른 번호가 나오도록 시작 값을 설정합니다.	5회 반복 하나의 주첨 결과를 만든 뒤 동일한 절차를 네 번 더 수행합니다.	정수 6칸 한 회차의 번호를 담는 저장 공간이며, 모든 함수가 같은 배열을 공유합니다.

사용자가 확인하는 결과는 정렬된 번호 6개이지만, 프로그램 내부에서는 난수 생성 → 중복 판정 → 저장 → 정렬 → 출력의 절차가 순서대로 진행됩니다. 한 회차에서는 생성 함수가 새로운 난수를 만든 뒤, 중복 판정 함수가 해당 번호가 이미 배열에 저장되어 있는지 확인합니다. 중복이면 번호를 저장하지 않고 다시 주첨하며, 중복이 아니면 현재 배열 위치에 저장합니다. 이 과정을 반복하여 서로 다른 번호 6개가 완성되면 정렬 함수와 출력 함수가 차례대로 실행됩니다.

프로그램의 반복 구조는 두 단계로 구성됩니다. 먼저 main() 함수의 for 문은 완성된 주첨 결과를 총 5회 출력하기 위한 전체 반복을 담당합니다. 그 안에서 getLottoNo() 함수는 배열의 0번 위치부터 5번 위치까지 번호를 채우며, 중복 번호가 생성되면 같은 위치에서 다시 번호를 주첨합니다. 따라서 조건을 통과한 번호만 배열에 저장되고 다음 처리 단계로 전달됩니다.

배열은 한 회차의 번호 6개를 저장하는 공간입니다. 번호 생성 단계에서는 유효한 값이 하나씩 저장되고, 정렬 단계에서는 같은 배열 안에서 숫자의 위치가 오름차순으로 재배치됩니다. 출력 단계에서는 정렬이 완료된 배열을 읽어 화면에 표시합니다. 이처럼 하나의 배열이 각 함수에 전달되면서 결과를 유지하고, 각 함수는 자신에게 주어진 기능만 수행하도록 구성하였습니다.

이 구조의 핵심은 잘못된 번호를 저장하기 전에 먼저 검사한다는 점입니다. 중복된 값은 즉시 재추첨 되므로, 정렬 함수와 출력 함수는 이미 조건을 만족한 6개의 번호만 처리합니다. 따라서 프로그램의 전체 흐름이 명확해지고, 범위·중복·정렬·출력 횟수라는 기준에 따라 실행 결과를 확인할 수 있습니다.

2.3) 함수 void getLottoNo(int iLottoArr[])와

int isDuplicateNo(const int iLottoArr[], int iIndex, int iRandNo)

중복 없는 번호를 생성하는 함수입니다. 이미 뽑힌 번호인지 매번 확인하여 한 회차 안에서 같은 번호가 반복 되는 오류를 방지합니다. 배열에 무작위 번호를 생성해서 저장하는 기본 로직에, 이미 뽑은 번호는 저장하지 않고 다시 뽑는 중복 방지 로직을 걸었습니다.

함수 | 번호 생성과 중복 판정의 역할 분리



- ① 생성 함수는 main()에서 선언된 1차원 배열 iLottoArr[]을 매개변수로 전달받습니다.
- ② 생성 함수에서는 rand() 함수로 난수를 생성하여 iRandNo에 저장합니다.
- ③ 생성 함수는 중복 판정 함수에 배열, 저장하려는 위치를 나타내는 인덱스, 새로 생성한 난수를 매개값으로 전달합니다. 판정 함수는 배열에 이미 저장된 값을 확인하기만 하므로, 해당 함수 내부에서 배열 요소를 변경하지 못하도록 const 배열 매개변수로 선언하였습니다.
- ④ 중복 판정 함수가 TRUE를 반환하면 생성 함수는 해당 번호를 저장하지 않고 재추첨하며, FALSE를 반환하면 배열의 해당 인덱스에 번호를 저장합니다. continue 문을 사용하여 중복 번호가 발생한 경우 while 문의 다음 반복으로 이동하고, 새로운 난수 생성을 다시 시작합니다.
- ⑤ 생성 함수는 void형이므로 반환 값은 없습니다. 배열은 함수에 전달될 때 배열의 시작 주소(포인터)가 전달됩니다. 따라서 함수 내부에서 배열 요소의 값을 변경하고 배열을 반환하지 않더라도 원본 배열에도 그대로 반영됩니다.

[중복 판정 호출 구조 요약]

```
iRandNo = (rand() % MAX_LOTTO_NO) + 1; //난수 생성
if (isDuplicateNo(iLottoArr, iIndex, iRandNo) == TRUE){
    continue; // 중복이면 재추첨
}
iLottoArr[iIndex] = iRandNo; //통과한 번호만 저장

int isDuplicateNo(const int iLottoArr[], int iIndex, int iRandNo) {
    int blsDuplicate = FALSE;
    for (int iCheckIndex = 0; iCheckIndex < iIndex; ++iCheckIndex){
        if (iRandNo == iLottoArr[iCheckIndex]){
            blsDuplicate = TRUE;
            break;
        }
    }
    return blsDuplicate;
}
```

2.4) 함수 void sortArrAsc(int iLottoArr[])

sortArrAsc() 함수는 추출이 완료된 6개의 로또 번호를 작은 수부터 큰 수 순서로 재배치하는 기능을 담당합니다. 로또 번호는 난수로 생성되기 때문에 처음 배열에 저장될 때는 순서가 일정하지 않습니다. 따라서 정렬된 형태로 출력하면 과제의 요구사항을 충족할 뿐 아니라, 사용자가 결과를 빠르게 확인할 수 있습니다.

본 프로그램에서는 강의 시간에 학습한 선택정렬(Selection Sort) 방식을 사용하였습니다. 선택정렬은 아직 정렬되지 않은 영역에서 가장 작은 값을 찾은 뒤, 현재 위치의 값과 교환하여 배열의 앞쪽부터 순서를 확정하는 방식입니다.

배열 · 정렬 번호 6개를 작은 값부터 정렬

정렬 전

31

7

42

15

3

28

[0]

[1]

[2]

[3]

[4]

[5]

정렬 후

3

7

15

28

31

42

[0]

[1]

[2]

[3]

[4]

[5]

현재 최솟값 3

오름차순 완성

선택 정렬 : 남은 값 중 가장 작은 값을 찾아 앞자리와 교환

배열 : 0번 인덱스부터 저장 | sortArrAsc() : 같은 배열 공간의 값을 오름차순으로 재배치

단계	탐색 시작 배열	Current Min	Found Min	Swap
1 Pass	31 7 42 15 3 28	31 [0]	3 [4]	0 교환
2 Pass	3 7 42 15 31 28	7 [1]	7 [1]	X 유지
3 Pass	3 7 42 15 31 28	42 [2]	15 [3]	0 교환
4 Pass	3 7 15 42 31 28	42 [3]	28 [5]	0 교환
5 Pass	3 7 15 28 31 42	31 [4]	31 [4]	X 유지

※ 배열 인덱스는 0부터 시작 → 값 3의 위치는 [4]

정렬 완료

3 7 15 28 31 42

오름차순 완성

코드에서 iLoop는 현재 값을 확정할 위치를 나타내며, iMin과 iMinIndex는 탐색 중에 발견한 최솟값과 그 위치를 저장합니다. 이후 더 작은 값이 발견되면 두 변수를 갱신하고, 탐색이 끝난 뒤 교환이 필요한 경우에는 iTemp를 사용하여 두 값을 안전하게 바꿉니다. 이를 통해 배열, 인덱스, 반복문, 비교문, 임시 변수를 이용한 값 교환 과정을 종합적으로 실습했습니다.

선택정렬은 데이터 수가 많아질수록 비교 횟수가 증가한다는 단점이 있습니다. 그러나 본 프로그램은 한 회차 당 6개의 번호만 정렬하므로 성능상의 부담이 거의 없으며, 정렬 과정을 이해하고 설명하기 쉽다는 점에서 과제 목적에 적합합니다. 또한 이 함수는 반환형이 void이지만, 매개변수로 전달된 배열이 main()에서 선언된 배열의 저장 공간을 가리키므로 정렬 결과는 원본 배열에 그대로 반영됩니다.

2.5) 함수 main()과 void prnArr(const int iLottoArr[], int iLoop)

prnArr() 출력 함수는 완성된 번호가 담긴 배열을 매개변수로 받아서 화면 형식에 맞게 출력합니다. 반복문을 통해 배열의 각 인덱스를 순회하며 원소 하나씩 출력할 수 있습니다. 출력 함수가 여러 회차에서 호출되는 점을 고려하여, iLoop를 추가 매개변수로 전달받고 '몇 번째 로또 번호'인지를 함께 출력하도록 작성하였습니다. 사용자가 보기 편하도록 슬래시(/)로 번호를 분리했습니다.

main() 함수는 프로그램의 진입점으로 작성한 함수들을 필요에 따라 불러와 호출합니다. 번호 생성이 완료된 뒤 정렬 함수와 출력 함수를 순서대로 호출하므로, 사용자는 오름차순으로 정리된 추첨 결과를 확인할 수 있습니다.

```

srand(time(0)); // 현재 시간 기반 난수 시드 설정
int iLottoArr[ARR_SIZE] = {0};
// 루틴 총 반복
for (int iLoop = 0; iLoop < LOTTO_REPEAT_COUNT; ++iLoop) { // 하나의 루틴
    // 로또 번호 생성
    getLottoNo(iLottoArr);
    // 로또 번호 정렬
    sortArrAsc(iLottoArr);
    // 로또 번호 출력
    prnArr(iLottoArr, iLoop);
}

```

난수 생성과 시드 설정에 필요한 함수 선언을 사용하기 위해 <stdlib.h>와 <time.h> 헤더 파일을 프로그램에 포함하였습니다. <stdlib.h>에는 rand() 함수와 srand() 함수의 선언이 포함되어 있으며, <time.h>에는 현재 시간 값을 얻기 위한 time() 함수의 선언이 포함되어 있습니다.

srand(time(0))는 현재 시간을 기준으로 난수열의 시작값인 시드를 설정하는 구문입니다. 실행 시점에 따라 시드 값이 달라질 수 있으므로, 프로그램을 다시 실행할 때 서로 다른 추첨 결과를 얻을 수 있습니다. 이후 번호 생성 함수에서는 (rand() % MAX_LOTTO_NO) + 1 연산을 사용하여 1부터 45 사이의 번호를 생성합니다.

3. 프로그램 실행 결과 및 검증

3.1) 실행화면

소스를 Visual Studio Community 2022에서 컴파일하고 두 차례 실행하여 결과를 확인하였습니다. 각 실행 결과에서 5개 회차가 출력되었으며, 각 회차의 번호는 1~45 범위 안에서 중복 없이 생성되고 오름차순으로 정렬되어 있음을 확인하였습니다.

```

===== 1번째 로또 번호 =====
{ 19 / 26 / 30 / 36 / 37 / 39 }

===== 2번째 로또 번호 =====
{ 7 / 11 / 14 / 16 / 26 / 37 }

===== 3번째 로또 번호 =====
{ 1 / 14 / 22 / 27 / 28 / 29 }

===== 4번째 로또 번호 =====
{ 18 / 24 / 26 / 34 / 38 / 39 }

===== 5번째 로또 번호 =====
{ 4 / 7 / 16 / 28 / 31 / 35 }

D:\Clang26-1\Prj_Exam\x64\Debug\Prj_Exam.exe(프로세스 35960)이(가) 0 코드(0x0)와 함께 종료되었습니다.
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용
하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...

```

```

===== 1번째 로또 번호 =====
{ 4 / 9 / 12 / 14 / 21 / 36 }

===== 2번째 로또 번호 =====
{ 2 / 5 / 19 / 26 / 37 / 40 }

===== 3번째 로또 번호 =====
{ 8 / 9 / 11 / 13 / 30 / 33 }

===== 4번째 로또 번호 =====
{ 5 / 8 / 22 / 29 / 34 / 42 }

===== 5번째 로또 번호 =====
{ 11 / 17 / 22 / 29 / 34 / 39 }

D:\Clang26-1\Prj_Exam\x64\Debug\Prj_Exam.exe(프로세스 32468)이(가) 0 코드(0x0)와 함께 종료되었습니다.
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용
하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...

```

3.2) 실행 결과 기준 요구 조건 확인

출력 횟수	5/5
번호 범위	정상 (모든 값 1~45)
중복 번호	회차별 중복 없음
정렬 상태	오름차순 정렬 완료

4. 결론

4.1) 확장 아이디어

① 사용자 로또 번호 입력 및 당첨 결과 비교 기능

: 선택 번호와 추첨 번호를 비교하고 일치 개수를 계산하는 로직 필요

② 추첨 결과 통계 기능

: 반복 추첨 결과(난수 번호 출현 빈도) 구하기, 출현 횟수 카운트 로직 이용

4.2) 학습 성과

이번 과제를 수행하기 전에 C++에서 클래스를 중심으로 객체지향 프로그래밍을 구성해 본 경험이 있었습니다. C++에서는 하나의 객체가 가져야 할 데이터는 멤버 변수로, 그 데이터를 처리하는 기능은 멤버 함수로 묶어 관리할 수 있었고, 클래스의 선언과 구현을 헤더 파일과 소스 파일로 분리하여 작성했습니다. 이 과정에서 프로그램을 하나의 큰 코드로만 작성하는 것이 아니라, 서로 관련 있는 데이터와 기능을 묶고 역할에 따라 파일을 분리해 모듈화하는 것이 코드의 구조를 이해하고 수정하는 데 도움이 된다는 점을 학습한 바 있습니다.

이번에는 C언어로 로또 복권 번호 추출 프로그램을 작성하면서, 객체와 클래스가 없는 환경에서는 기능을 중심으로 프로그램을 어떻게 분리할 수 있는지 실습하였습니다. main() 함수에 모든 처리 과정을 작성하는 대신, 번호 생성 기능, 중복 판정 기능, 오름차순 정렬 기능, 결과 출력 기능을 각 별도의 함수가 담당하도록 분리하였습니다. 또한 함수의 선언은 헤더 파일에 작성하고, 실제 구현은 별도의 소스 파일에 정의하였으며, main() 함수에서는 구현된 함수들을 필요한 순서대로 호출하도록 구성하였습니다. 이를 통해 C언어에서도 여러 함수를 하나의 논리 모듈, 작은 라이브러리처럼 분리하여 관리할 수 있음을 이해했습니다.

특히 처음에는 번호를 생성하는 함수 안에 중복 판정 로직까지 함께 작성하였지만, 프로그램을 다시 살펴보면 서로 '번호를 생성하는 기능'과 '이미 존재하는 번호인지 판단하는 기능'은 서로 다른 책임을 가진다고 판단하여 둘을 분리하였습니다. 하나의 함수가 하나의 분명한 역할을 담당하도록 구조를 개선한 과정이었습니다. 기능 분리와 모듈화는 코드의 가시성을 높이는 데 그치지 않고, 프로그램의 의도를 더 명확하게 표현하는 설계 방식이라는 것을 체감할 수 있었습니다.

다만 과제를 진행하면서 가장 어렵게 느껴진 부분도 기능을 어디까지 하나의 함수로 묶고, 어느 시점에서 별도의 함수로 분리해야 하는지 판단하는 일이었습니다. 제출할 결과물이라는 관점에서 코드를 반복해서 살펴보니, 단순히 동작하는 코드보다 각 기능의 책임이 명확하게 보이는 코드가 더 좋은 구조라는 생각이 들었지만 아직 프로그램을 어떤 기준으로 얼마나 세밀하게 모듈로 나누어야 하는지에 대한 많은 고민이 남았습니다. 하지만 좋은 설계에 대해 고민하는 시각은 이전보다 조금씩 넓어지고 있다고 느꼈습니다.

또한 이번 과제를 단순히 소스 코드를 제출하는 것으로 끝내지 않고, 프로그램을 잘 모르는 사람도 이해할 수 있도록 하나의 보고서 형태로 정리하는 과정 역시 의미가 컸습니다. 프로그램을 작성할 때에는 자연스럽게 사용했던 구문이나 용어들을 보고서에 설명하려면 각각의 정확한 개념을 다시 확인해야 했습니다. 이번에는 시각적인 요소를 특히 신경 써서 작성해 보았는데, 제가 작성한 코드가 실제로 어떤 순서와 이유로 동작하는지 그 흐름을 다시 한번 점검할 수 있었습니다.

결과적으로 이번 로또 복권 번호 추출 프로그램은 지금까지 수업에서 배운 개별 문법과 개념을 하나씩 사용하는 단계를 넘어, 이를 하나의 작은 프로젝트 안에서 연결하여 구현해 본 경험이었습니다. 각 주차에서 배운 내용을 종합적으로 적용해 보며 C언어 문법에 대한 이해뿐 아니라, 프로그램을 기능별로 나누어 설계하고, 역할을 명확히 하며, 작성한 결과물을 다른 사람이 이해할 수 있도록 정리하는 과정의 중요성을 다시 한번 새길 수 있었습니다.